

# Verificação e Validação de Software

## Trabalho 1: Defeitos Encontrados na Primeira Versão

Eduardo Alcária  
Arthur Pimentel  
Ethan Soares

### Contexto

A primeira versão da classe `CalculadoraTarifa` foi submetida ao conjunto completo de 25 casos de teste unitários (arquivo `CalculadoraTarifaTest.java`, técnica de particionamento e valor limite). A execução resultou em 8 casos com falha, agrupados em 4 defeitos distintos, descritos a seguir.

### Defeito 1: cortesia com limite exclusivo

#### Falha observada

No limite exato de 20 minutos de permanência, a implementação cobrava a tarifa fixa de R\$ 15,00 em vez de aplicar a cortesia (valor esperado R\$ 0,00).

#### Caso de teste que detectou

CT02 (entrada 10/03/2026 08:00, saída 10/03/2026 08:20, sem VIP). Resultado obtido: R\$ 15,00. Resultado esperado: R\$ 0,00.

#### Causa

A condição de cortesia usava comparação estrita:

```
if (minutos < MINUTOS_CORTESIA) {  
    return 0.0;  
}
```

O enunciado define a cortesia como “20 minutos de cortesia”, ou seja, o limite de 20 minutos deve estar incluído na faixa de valor zero.

#### Correção proposta

Trocar o operador de comparação para incluir o limite:

```
if (minutos <= MINUTOS_CORTESIA) {  
    return 0.0;  
}
```

### Defeito 2: hora adicional cobrada apenas quando completa

#### Falha observada

Para permanências que ultrapassam a primeira hora sem completar um múltiplo exato de 60 minutos, o valor cobrado ficava R\$ 5,00 abaixo do esperado, pois a hora adicional iniciada não era cobrada.

## Casos de teste que detectaram

- CT06 (61 minutos): obtido R\$ 15,00, esperado R\$ 20,00.
- CT08 (121 minutos): obtido R\$ 20,00, esperado R\$ 25,00.
- CT12 (179 minutos): obtido R\$ 20,00, esperado R\$ 25,00.
- CT13 (359 minutos): obtido R\$ 35,00, esperado R\$ 40,00.

## Causa

O número de horas adicionais era calculado por divisão inteira, que trunca o resultado em vez de arredondar para cima:

```
long horasAdicionais = minutosExcedentes / MINUTOS_TARIFA_FIXA;
```

O enunciado especifica que o incremento ocorre “a cada intervalo de 1 hora (inclusive)”, ou seja, qualquer minuto iniciado dentro de uma nova hora já deve gerar a cobrança daquele intervalo.

## Correção proposta

Arredondar o número de horas adicionais para cima:

```
long horasAdicionais = (minutosExcedentes + MINUTOS_TARIFA_FIXA - 1) /  
    MINUTOS_TARIFA_FIXA;
```

## Defeito 3: pernoite não soma múltiplas diárias

### Falha observada

Quando o veículo permanece por mais de uma pernoite, o sistema cobrava apenas uma diária de R\$ 50,00, independentemente do número de noites em que o veículo permaneceu estacionado.

### Caso de teste que detectou

CT15 (entrada 10/03/2026 20:00, saída 12/03/2026 08:00, duas pernoites completas). Resultado obtido: R\$ 50,00. Resultado esperado: R\$ 100,00.

## Causa

A condição de pernoite era tratada como um valor booleano único, aplicando sempre o mesmo valor fixo:

```
boolean pernoite = saida.toLocalDate().isAfter(entrada.toLocalDate())  
    && !saida.toLocalTime().isBefore(ABERTURA);  
double valor = pernoite ? VALOR_PERNOITE : calcularPorPermanencia(minutos)  
    ;
```

Isso não reflete o caso de um veículo que permanece por dois ou mais dias, no qual o valor de pernoite deve ser cobrado uma vez para cada janela de 24 horas excedida.

## Correção proposta

Contar quantas vezes a janela das 08:00 do dia seguinte é ultrapassada e multiplicar pelo valor unitário de pernoite:

```
long contarPernoites(LocalDateTime entrada, LocalDateTime saida) {  
    LocalDateTime limite = entrada.toLocalDate().plusDays(1).atTime(  
        ABERTURA);  
    long pernoites = 0;
```

```

    while (!saida.isBefore(limite)) {
        pernoites++;
        limite = limite.plusDays(1);
    }
    return pernoites;
}

```

## Defeito 4: janela de saída proibida com limites exclusivos

### Falha observada

Uma saída registrada exatamente às 02:00 ou exatamente às 07:59 era aceita pelo sistema, quando deveria ser rejeitada por cair dentro do intervalo em que a saída de veículos não é permitida.

### Casos de teste que detectaram

- ES01 (saída às 02:00 do dia seguinte à entrada): esperava-se `IllegalArgumentException`, porém nenhuma exceção foi lançada.
- ES02 (saída às 07:59 do dia seguinte à entrada): esperava-se `IllegalArgumentException`, porém nenhuma exceção foi lançada.

### Causa

A verificação usava comparações estritas em ambas as extremidades do intervalo:

```

boolean dentroJanelaProibida = hora.isAfter(INICIO_SAIDA_PROIBIDA)
    && hora.isBefore(FIM_SAIDA_PROIBIDA);

```

O enunciado define o intervalo proibido como “das 02:00 da manhã até as 07:59 da manhã”, portanto ambos os extremos devem ser considerados parte do intervalo proibido.

### Correção proposta

Tornar a comparação inclusiva nos dois limites:

```

boolean dentroJanelaProibida = !hora.isBefore(INICIO_SAIDA_PROIBIDA)
    && !hora.isAfter(FIM_SAIDA_PROIBIDA);

```

## Observação adicional sobre o desenho dos testes

Durante a análise, os casos ES01 e ES02 foram inicialmente escritos com a saída no mesmo dia da entrada, o que fazia a validação de ordem cronológica (saída deve ser posterior à entrada) mascarar o defeito real da janela de saída proibida. Os casos foram reformulados posicionando a entrada no dia anterior, isolando corretamente a condição sob teste.

## Resultado após a correção

Após aplicar as quatro correções descritas, a suíte completa de 25 casos de teste passou a executar com sucesso (25 de 25), confirmando que os defeitos identificados na primeira versão foram sanados sem introduzir regressões nos demais casos.